Systolic Array 기반 Pointwise Convolution 엔진 설계 및 검증 보고서

(MobileNetV2 Layer08 가속기 — 1단계)

Weight-Stationary 8×8 Systolic Array,

고정소수점 C 모델 대비 비트 단위 검증

과 정 명	ETRI RTL 취업연계 교육
작 성 자	박준영
제 출 일	2026. 08. 02.

목 차

1. 개요 4

1.1 설계 배경 4

1.2 이번 제출 범위 4

1.3 검증 완료 요약 4

2. 시스템 사양 5

2.1 대상 보드 및 클록 5

2.2 데이터 포맷 5

2.3 곱셈기 및 메모리 5

2.4 초기화 데이터(coe/hex) 생성 5

3. 용어 정의 6

3.1 용어 정의 표 6

3.2 타일 처리 루프 순서 6

4. 설계 구조 8

4.1 모듈 계층 8

4.2 Weight-Stationary 매핑 및 타일링 9

4.3 타일 처리 시퀀스 및 전체 소요 9

5. 동작 원리 및 설계 결정 10

5.1 Weight-Stationary 선택 10

5.2 지연이 있는 연산기의 systolic 통합 10

5.3 누산 버퍼(Accum_Buffer) 설계 10

5.4 제어 스타일 10

6. 검증 전략 (2단 골든 체계) 11

6.1 1단(정확도): float C → 고정소수점 C 모델 11

6.2 2단(기능): RTL vs 고정소수점 C 모델 — 비트 단위 일치 11
6.3 디버그 데이터셋 방법론 11
7. 파형 분석 (디버그 데이터셋, 최종 성공본) 12
7.1 전체 파형 (0~871μs) 12
7.2 LOAD → CALC 진입 12
7.3 부분합 도착 · CALC 종료 · 타일 전환 13
7.4 분석 요점 15
8. 문제 해결: BMG 출력 레지스터로 인한 "정확히 7/8" 오답 16
8.1 증상 16
8.2 원인 분석 16
8.3 해결 16
8.4 고찰 17
9. 결론 및 고찰 18
9.1 결론 18
9.2 고찰 18
9.3 다음 단계 18

1. 개요

1.1 설계 배경

- 최종 프로젝트는 MobileNetV2의 Inverted Residual Bottleneck(Layer08: 64ch→384ch→64ch, 14×14)을 FPGA 가속기로 구현하는 것임.
- 전체 연산의 약 90%가 1×1 pointwise conv에 집중됨(확장 482만 MAC + 축소 482만 MAC vs DW 68만 MAC).
- 따라서 NPU 공부를 하며 배웠던 Systolic Array를 1단계로 pointwise conv 엔진에 systolic array를 적용하여 설계·검증함.

1.2 이번 제출 범위

- PW 확장 단계(64ch→384ch)를 대상으로 함.
- 입력 64채널×14×14(Q3.12), 1×1 필터 384개(가중치 24,576개), 출력 384채널×14×14 = 75,264개 값임.

1.3 검증 완료 요약

- 이미 검증된 float C 골든 모델로부터 고정소수점(int16) C 모델을 파생함(불일치 0/12,544, 최대 오차 0.0013).
- RTL 최종 출력 75,264개 전부를 고정소수점 C 모델과 비트 단위 일치로 확인함.
- Vivado xsim에서도 동일하게 PASS함.

2. 시스템 사양

2.1 대상 보드 및 클록

- 대상 보드는 Ultra96 v2(Zynq UltraScale+ ZU3EG, DSP 360개, BRAM 7.6Mb)임.
- 시스템 클록은 100MHz임.

2.2 데이터 포맷

- 고정소수점 포맷은 실측 데이터 기반으로 설계함.

항목	포맷	실측 근거 / 마진
입력 활성	Q3.12 signed 16b	Layer07 출력 실측 -1.77~+1.98, 마진 3.4배
가중치	2^-16 스케일 16b	BN 폴딩 후 |w|max 0.142, 마진 3.5배
누산기	32b signed (f=28)	실측 최대 29비트 + 3비트 가드

2.3 곱셈기 및 메모리

- 곱셈기: Vivado Multiplier IP(mult_gen_0), 16×16 signed → 32b, Pipeline Stages 4를 사용함(곱 결과가 4클럭 뒤 출력, MUL_LAT=4).
- 메모리: Vivado BMG Simple Dual Port 2개를 사용하고 coe로 초기화함.
- 읽기 지연은 1클럭(MEM_LAT=1, Primitives Output Register OFF)임.
- Primitives Output Register 옵션 관련 디버깅 사례는 8장에서 서술함.

메모리	구성	워드 구성 / 주소
A (입력)	128b × 1,568	워드 = 같은 위치의 8채널 패킹, 주소 = ch_tile×196 + 위치
B (가중치)	128b × 3,072	워드 = 한 행(8필터) 묶음, shift-in을 위해 행 역순 저장, 주소 = (filter_tile×8 + ch_tile)×8 + load_cnt

2.4 초기화 데이터(coe/hex) 생성

- coe/hex는 C 유틸(make_input_coe.c / make_weight_coe.c)로 생성함.
- float 데이터를 round-half-up으로 양자화(고정소수점 C 모델과 동일 규칙)하고 패킹 규칙대로 재배열함.

3. 용어 정의

3.1 용어 정의 표

- 보고서를 처음 읽는 사람이 이후 장을 읽는 데 필요한 용어를 아래 표 하나로 정리함.

용어	정의	이 설계에서의 수치
위치(픽셀)	14×14 출력 평면 위의 한 점. 파형의 out_pos	196개 (0~195)
채널	픽셀 196개가 모인 평면 한 장. 필터 1개가 출력 채널 1장을 만듦	입력 64장, 출력 384장
필터	출력 채널 1장을 만드는 1×1×64 가중치 묶음 (채널당 1개씩 64개 숫자)	384개, 가중치 총 24,576개
타일	가중치 행렬(64행×384열)을 PE 배열 크기(8×8)로 자른 조각. 채널 8개 × 필터 8개 = 가중치 64개	8(채널 방향)×48(필터 방향) = 384조각
창 (타일 처리 1회)	타일 1개를 처리하는 LOAD(8)+CALC(196)+DRAIN(19) 구간. 파형에서 뱅크 valid가 켜졌다 꺼지는 단위	≈227클럭, 총 384회
부분합	64항 내적 중 8항(채널 타일 1개분)씩만 더한 중간값	디버그셋 기준 ±0.25/0.375/0.5/0.625
완성값	채널 타일 8회가 모두 누적된 최종 내적 = 출력 1개	디버그셋 기준 ±2.0/3.0/4.0/5.0
뱅크	누산 버퍼의 조각. 열(필터)마다 1개, 깊이 196(위치 수)×32b. 필터 1장의 14×14 출력 지도를 8번에 나눠 그리는 저장소	8개 (BANK[0]~[7])
Weight-Stationary	가중치를 PE에 고정하고 입력·부분합이 흐르는 systolic 방식 (TPU v1과 동일 분류)	8×8, PE당 가중치 1개
skew / de-skew	입력 행 i를 i클럭 지연시키는 입력 계단 / 열 j 출력이 j클럭 늦는 계단을 뱅크 쓰기 주소로 흡수하는 것	행·열 각각 0~7클럭
valid 윈도우	열 j의 배열 출력이 유효한 구간. 시작 = PE_ROW(8)+MUL_LAT(4)+MEM_LAT(1)+j	[13+j, 13+j+196)

3.2 타일 처리 루프 순서

- 필터 타일이 바깥 루프, 채널 타일이 안쪽 루프임.
- 채널 타일 8회(창 8개)가 끝날 때마다 필터 8개의 출력이 동시에 완성·배출됨.

- 이것이 48회 반복되어 384필터가 완성됨.

4. 설계 구조

4.1 모듈 계층

모듈	역할
Layer08(top)	**시퀀서 FSM 골격 + A/B 메모리 인스턴스 + pw_engine 배선**
Pointwise_Conv(pw_engine)	**주소 생성 + 아래 서브모듈 통합**
Ctrl_FSM	**IDLE→LOAD(8)→CALC(196)→DRAIN(19) + 타일 루프(384회) + valid/카운터 생성**
Skew_Regs	**행 i를 i클럭 지연시키는 FF 체인 (입력 계단 정렬)**
SystolicArray	**8×8 PE 배선만 있는 껍데기 (제어 로직 없음)**
ProcessingElement ×64	**w_reg 고정 + mult_gen IP + psum 누산 전달**
Accum_Buffer	**8뱅크 × 196 × 32b 부분합 누산 (BRAM 추론)**

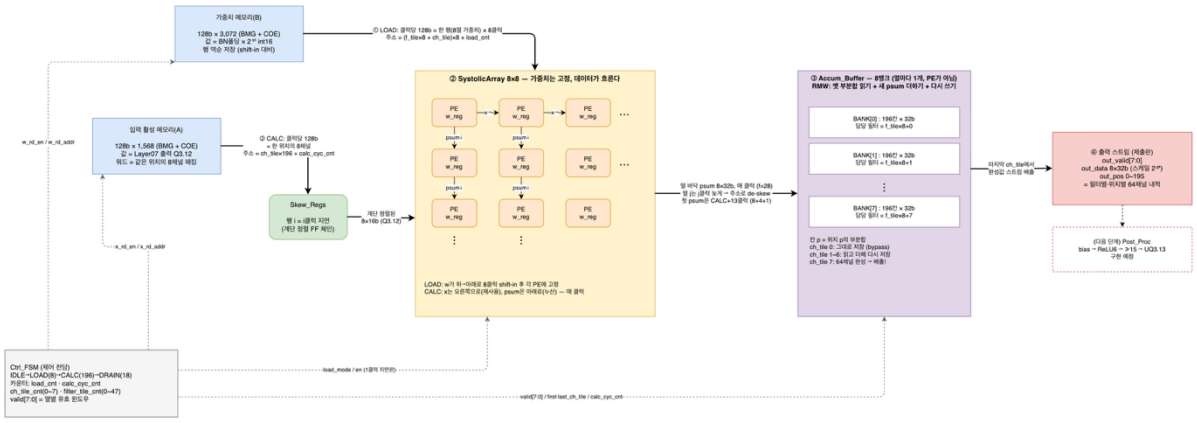


그림 1. 전체 데이터 흐름도

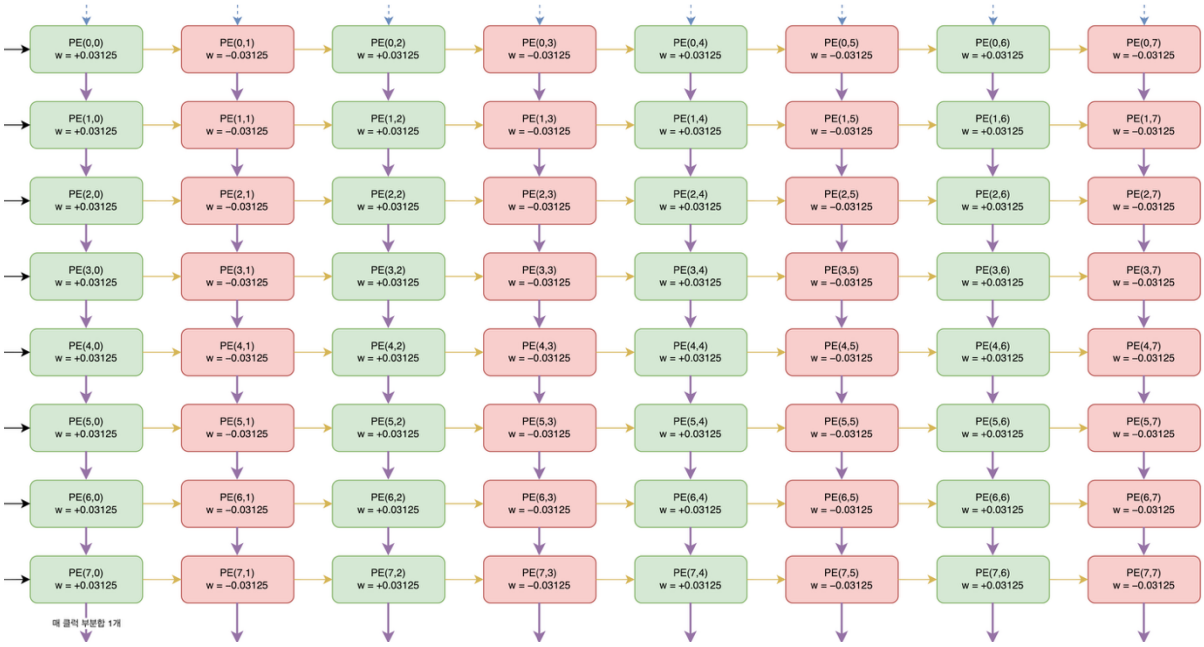


그림 2. 8×8 PE 배열과 필터 타일링

4.2 Weight-Stationary 매핑 및 타일링

- 행(8) = 입력 채널 축(누산 차원), 열(8) = 필터 축(병렬 차원)이며, PE(i,j)는 가중치 1개를 w_reg에 고정함.
- 입력 x는 좌→우로 흘러 8개 열이 재사용하고, 부분합 psum은 상→하로 내려가며 누산됨. 열 바닥에서 매 클럭 8채널 부분 내적이 완성됨.
- 타일링: 가중치 행렬(64행×384열)을 8×8 조각 384개(ch_tile 8 × filter_tile 48)로 분할함.
- 루프 순서는 filter_tile 바깥 / ch_tile 안쪽임. 채널을 안쪽에 둔 이유: 반대로 하면 384필터 전체의 미완성 부분합을 동시에 보관해야 하여 누산 버퍼가 48배 필요해짐. 현재 구조는 8뱅크로 충분함.

4.3 타일 처리 시퀀스 및 전체 소요

- 타일 1개의 처리(≈227클럭): LOAD 8클럭(가중치 64개 shift-in, 역순 공급) → CALC 196클럭(위치 0~195 스트리밍) → DRAIN 19클럭(파이프 배출)임.
- 8타일(ch_tile 0~7)마다 필터 8개의 14×14 출력이 동시에 완성·배출됨(last_ch_tile, out_valid). 이것이 48회 반복되어 384채널이 완성됨.
- 전체 소요는 87,169클럭 = 0.87ms @100MHz임(스펙 어림 482만 MAC ÷ 64 PE + 오버헤드와 부합).

5. 동작 원리 및 설계 결정

5.1 Weight-Stationary 선택

- 타일 하나에서 가중치 1개가 196개 위치에 재사용됨. 가장 많이 재사용되는 피연산자를 고정하는 것이 메모리 대역폭 측면에서 최적임.
- 스트리밍 공급이 입력 하나뿐이라 버퍼·제어도 단순함(Output-Stationary는 두 스트림 스큐 + 별도 drain 네트워크 필요).

5.2 지연이 있는 연산기의 systolic 통합

- 곱셈기 IP의 4클럭 지연(MUL_LAT=4)은 모든 행에서 동일하므로 행 간 정렬에서 상쇄됨. skew 구조는 그대로 두고, 배열 출력 타이밍만 상수 +4클럭이 됨.
- 메모리 읽기 1클럭(MEM_LAT=1)은 SA 제어신호(load_mode/en)의 1클럭 지연판으로 정렬함.
- 결과적으로 열 j의 유효 윈도우는 valid[j] = (calc_cyc_cnt ≥ 13+j) && (< 13+j+196)임. 13 = PE_ROW(8) + MUL_LAT(4) + MEM_LAT(1).
- 이 식이 파형 실측과 클럭 단위로 정확히 일치함(7장).

5.3 누산 버퍼(Accum_Buffer) 설계

- 행 타일링의 산물임: 64채널 내적을 8채널씩 8토막 내므로 미완성 부분합을 타일 사이에 보관해야 함. 뱅크 = 열마다 1개(총 8개), 깊이 196(위치 수) × 32b임.
- de-skew: 열 j의 출력이 j클럭 늦게 나오는 계단을 쓰기 주소 시작을 j클럭 늦춰 흡수함(별도 FF 역계단 불필요).
- RMW 2단 파이프(읽기 발행+psum 캡처 → 가산+쓰기)로 구성함. 주소가 매 클럭 단조 증가하고 같은 주소 재방문은 다음 채널 타일(196+α클럭 뒤)이므로 hazard 없음.
- 초기화는 first_ch_tile bypass(acc_new = first ? psum : old + psum)로 처리함. BRAM은 리셋 불가하므로 클리어 사이클 없이 첫 타일에 덮어쓰는 방식이 정석임.
- 마지막 타일(last_ch_tile)에서는 저장과 동시에 완성값을 스트림으로 배출함(융합). 32b 원시 결과 75,264개(2.4Mb)를 별도 저장하지 않는 이유임.

5.4 제어 스타일

- 상태 인코딩은 Ctrl_FSM 내부에만 두고 외부에는 제어 신호만 출력함(모듈 간 결합 최소화).
- 출력 신호는 n_state 기준 레지스터로 생성함(글리치 없음 + c_state와 동일 타이밍, PE 64개 팬아웃 대비).
- 카운터는 c_state 기준으로 생성함(상태 구간과 1:1 정렬).

6. 검증 전략 (2단 골든 체계)

6.1 1단(정확도): float C → 고정소수점 C 모델

- 이미 검증된 float C 골든 모델로부터 고정소수점 C 모델(fixp_layer08.c)을 파생함.
- BN 폴딩(ε=1e-5) + round-half-up 양자화 + 정수 conv + 재양자화 경로로 구성함.
- 결과: 판정 기준 1e-2에서 불일치 0/12,544, 최대 오차 0.0013, RMSE 0.00027임.
- 누산기 실측 최대 29비트로, 32b 대비 3비트 가드를 확보함.

[그림 3 삽입: 고정소수점 C 모델 실행 결과(불일치 0, 최대 오차 0.0013)]

그림 3. 고정소수점 C 모델 실행 결과

6.2 2단(기능): RTL vs 고정소수점 C 모델 — 비트 단위 일치

- 단위 TB 4종을 운용함: tb_PE(열 내적/역순 shift-in/가중치 고정), tb_SystolicArray(8×8 GEMM 80출력 전수), tb_Ctrl_FSM(6타일 축소 구성 사이클 채점 ×2회 재트리거), tb_Layer08(통합).
- 통합 TB(tb_Layer08)의 자가채점: RTL과 같은 hex를 TB도 직접 읽어 ① 첫 타일 psum 1,568개(196위치×8열)를 8채널 부분 내적과 대조, ② 최종 출력 75,264개 전부를 64채널 전체 내적과 대조함.
- 결과 errors=0 PASS, pw_done 87,169클럭임. Icarus Verilog(Mac)와 Vivado xsim 양쪽에서 동일함.

[그림 4 삽입: 통합 시뮬레이션 PASS 로그(75,264개 비트 일치)]

그림 4. 통합 시뮬레이션 PASS 로그

6.3 디버그 데이터셋 방법론

- 실데이터의 복잡한 소수 대신 암산 가능한 값으로 별도 coe/hex를 생성하여 파형 눈검사에 사용함.
- 입력 = 위치%4에 따라 1.0/1.5/2.0/2.5, 가중치 = 짝수 필터 +0.03125/홀수 -0.03125, 기대 출력 = ±2.0/3.0/4.0/5.0임.
- 타일마다 완성값의 1/8씩(예: 0.5→1.0→…→4.0) 쌓이는 누산 과정이 육안으로 확인됨.
- 파형 radix는 Fixed Point(입력 12, 가중치 16, 누산 28)로 실수 표시함.

7. 파형 분석 (디버그 데이터셋, 최종 성공본)

7.1 전체 파형 (0~871µs)

- start 1회로 시작해 done 1회로 종료함.
- TB의 기대값 계산 변수 ref_full이 진한 값 구간을 정확히 48번 보임 = 완성·배출 구간 48회(8타일 = 18.16µs 주기)임.
- 한 번에 필터 8개씩 완성되므로 48회 × 8 = 384필터 전부임. BANK last_d 펄스도 같은 주기로 48회임.

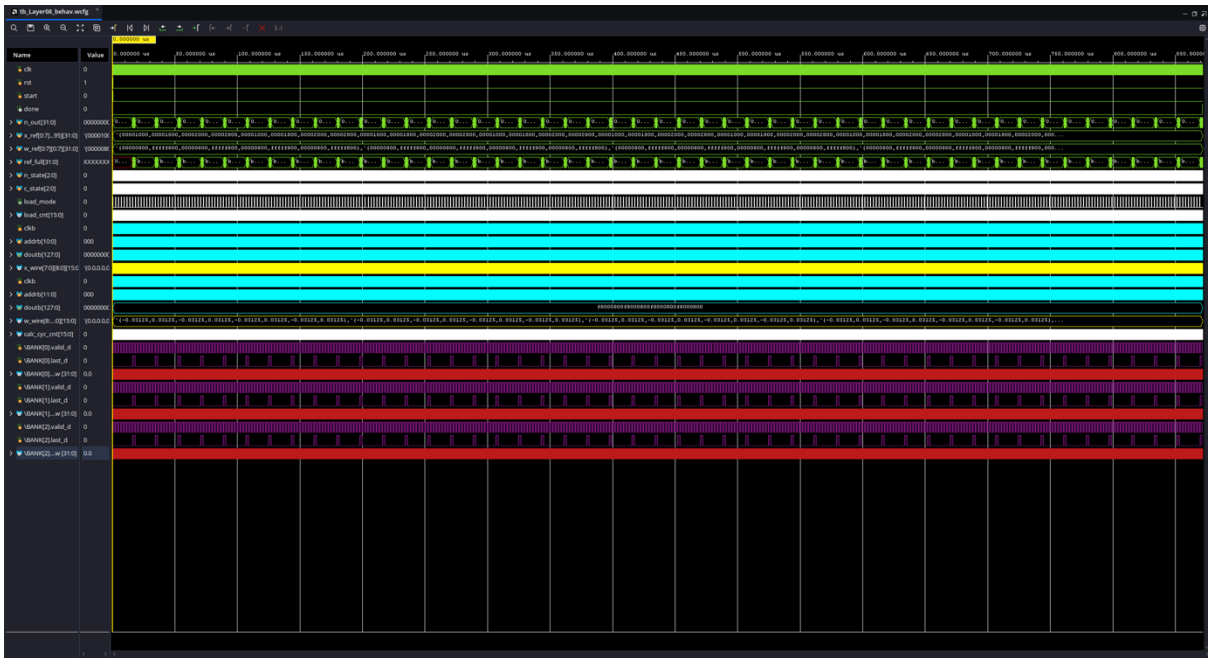


그림 5. 전체 파형 (완성·배출 구간 48회)

7.2 LOAD → CALC 진입

- LOAD 시작(35ns): start 펄스 후 load_mode 상승, load_cnt 0~7, B 메모리 주소 0~7 순차 읽기가 진행됨.
- doutb가 주소 제시 1클럭 뒤 f8000800...(±0.03125)으로 도착함(출력 레지스터 OFF 확인).
- 8클럭 뒤 w_wire[0]~[8] 전 행이 ±0.03125로 채워짐. 맨 아래 행(w_wire[8])까지 정상임.

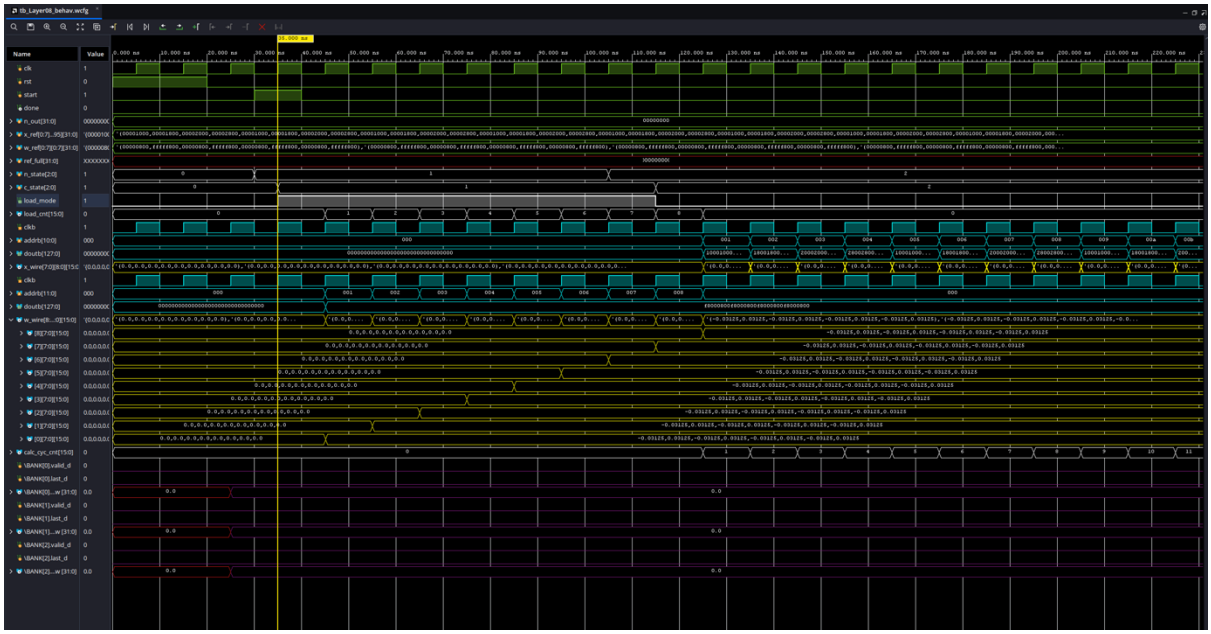


그림 6. LOAD 구간 (가중치 shift-in)

- CALC 시작: calc_cyc_cnt 0부터 증가하고, A 메모리 주소 = 위치 번호임.
- doutb에 1.0/1.5/2.0/2.5 디버그 입력 스트림이 보이고, x_wire에 행별 1클럭 지연(skew) 계단이 나타남.

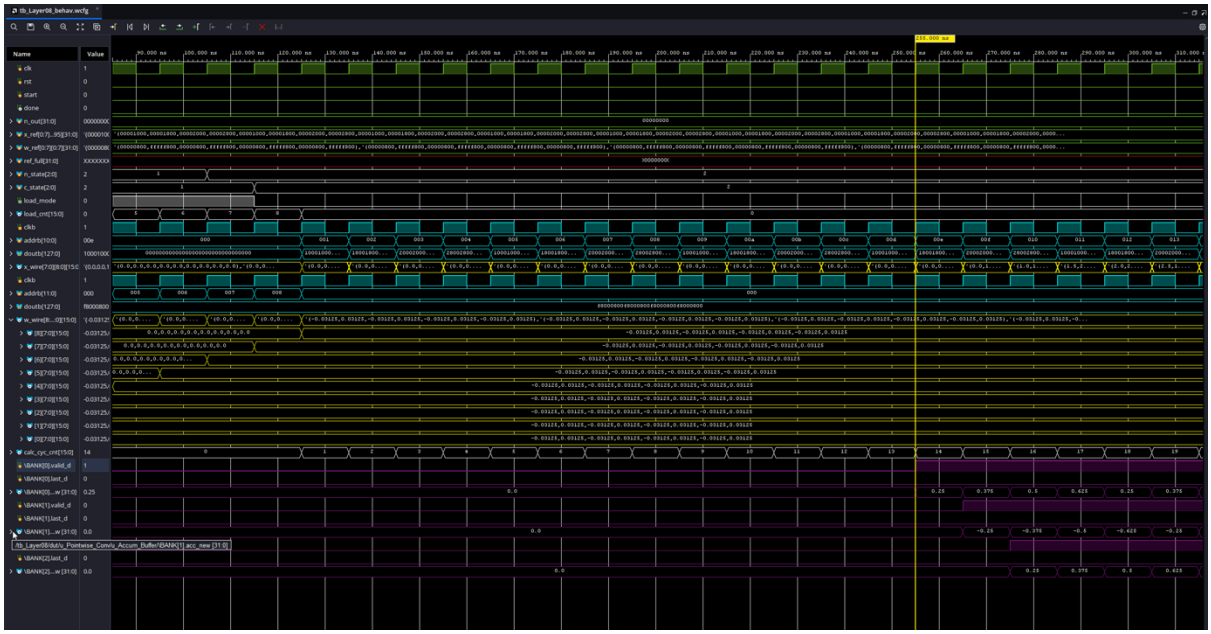


그림 7. CALC 시작 (입력 스트리밍과 skew)

7.3 부분합 도착 · CALC 종료 · 타일 전환

- 첫 부분합 도착: calc_cyc_cnt=14에서 BANK[0].valid_d 상승함(설계식 OFFSET 13 + 파이프 1클럭과 정확히 일치).
- BANK[0] acc_new = 0.25/0.375/0.5/0.625 반복(기대값과 일치), BANK[1]은 1클럭 뒤 −0.25/−0.375/...(홀수 필터 부호 반전), BANK[2]는 2클럭 뒤 — de-skew 계단임.

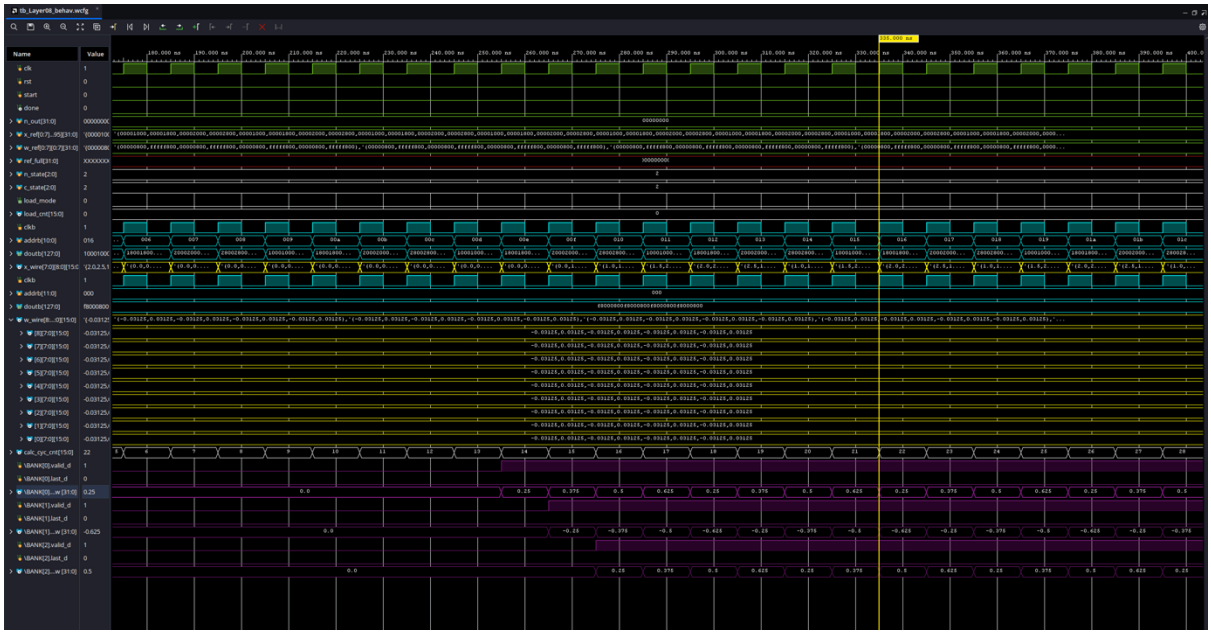


그림 8. 첫 부분합 도착과 de-skew 계단

- CALC 종료: 유효 구간이 정확히 196클럭(위치 196개) 만에 닫히고 DRAIN으로 천이함. 뱅크는 마지막 값을 유지함.

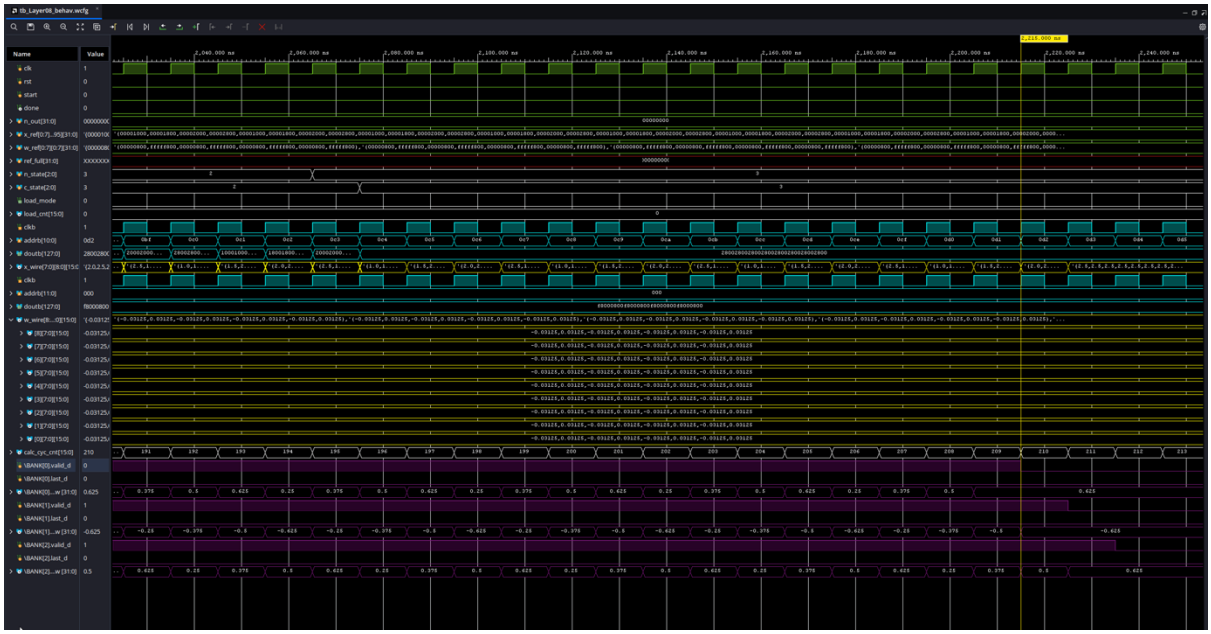


그림 9. CALC 종료와 DRAIN 천이

- 다음 타일 LOAD: cnt 218→0 리셋, ch_tile 0→1로 증가함.
- B 주소가 8~15(다음 채널 토막의 가중치)로 점프하고, A 주소는 196(다음 채널 묶음의 위치 0)부터 시작함.

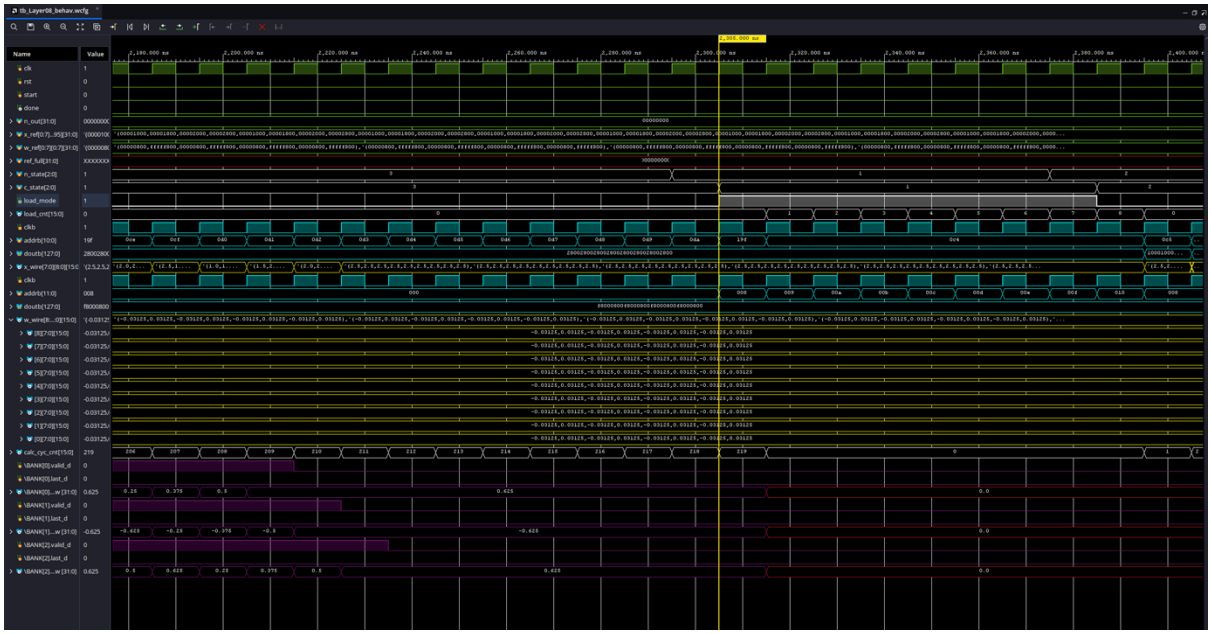


그림 10. 타일 전환 (ch_tile 0→1)

7.4 분석 요점

- ① valid[j] 시작 시점(13+j)이 5.2절의 식과 클럭 단위로 일치함.
- ② 8타일마다 last_ch_tile 구간에서 완성값 ±2.0/3.0/4.0/5.0이 배출되고, 그 시점의 B 주소가 0x40 단위로 점프함(다음 필터 묶음 9~16/384 진입).
- ③ 전체 파형의 48회 배출 주기가 "8타일 = 필터 8개 완성 × 48묶음 = 384필터" 구조를 한 장으로 증명함.

8. 문제 해결: BMG 출력 레지스터로 인한 "정확히 7/8" 오답

8.1 증상

- Vivado xsim 첫 실행에서 모든 누산값이 기대치의 정확히 7/8로 관측됨(0.25→0.21875, 0.375→0.328125, 0.5→0.4375, 0.625→0.546875).
- TB 전수 비교에서 전수 불일치가 발생함.

8.2 원인 분석

- ① 비율이 정확히 7/8 = "8행 중 1행 결손"의 산술적 서명임.
- ② 파형에서 w_wire[8](맨 아래 행 가중치 관찰점)이 0.0으로 고정됨을 발견함.
- ③ B 메모리 doutb가 읽기 시작 2클럭 뒤에야 데이터를 내는 것을 확인함.
- 원인: BMG의 Primitives Output Register가 ON(읽기 지연 2클럭)이었음 — 설계 전제는 1클럭임.
- LOAD의 첫 shift-in 에지에 데이터가 아직 없어 0이 들어가고, 마지막 가중치 워드는 shift 창이 닫힌 뒤 도착해 버려짐 → 맨 아래 행 가중치 전부 0.
- 입력 메모리도 같은 설정이어서 부가적으로 뱅크 시작이 1클럭 밀리는 서명(설계식 cnt14 대비 cnt15)도 관측됨.

8.3 해결

- 두 BMG 모두 Primitives Output Register 해제(읽기 지연 1클럭) 후 재생성함.
- 누산값이 기대값과 일치하고, TB errors=0, w_wire[8] 정상(±0.03125)임.

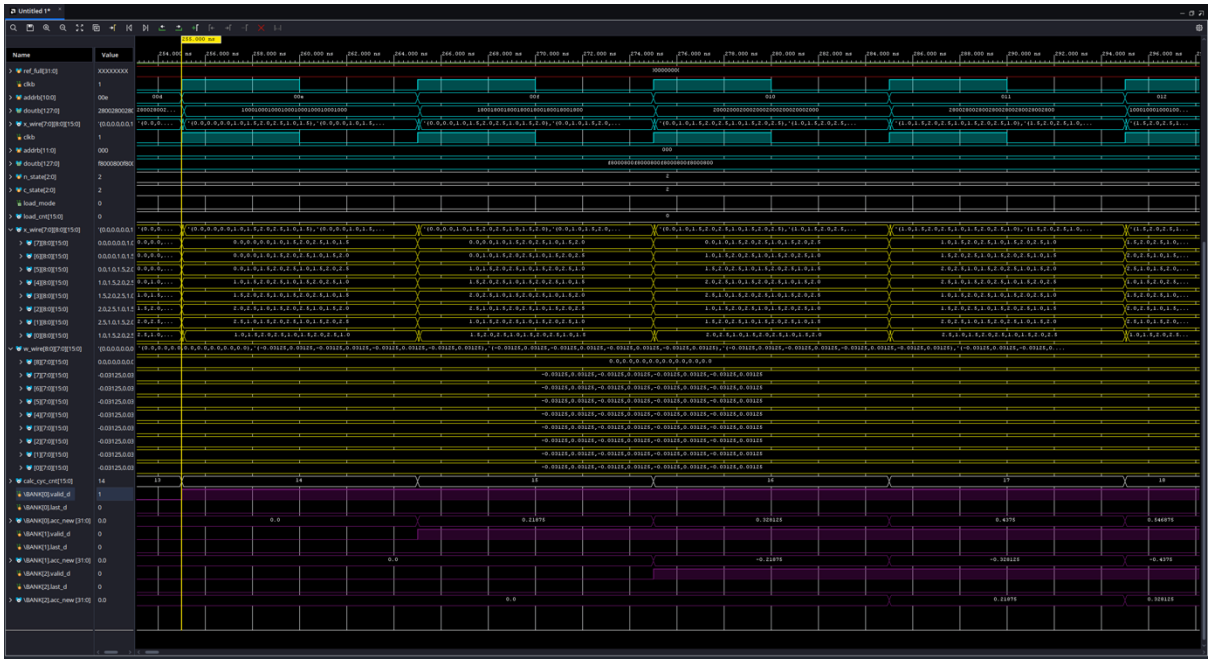


그림 11. 수정 전 파형 (7/8 배율 오답)

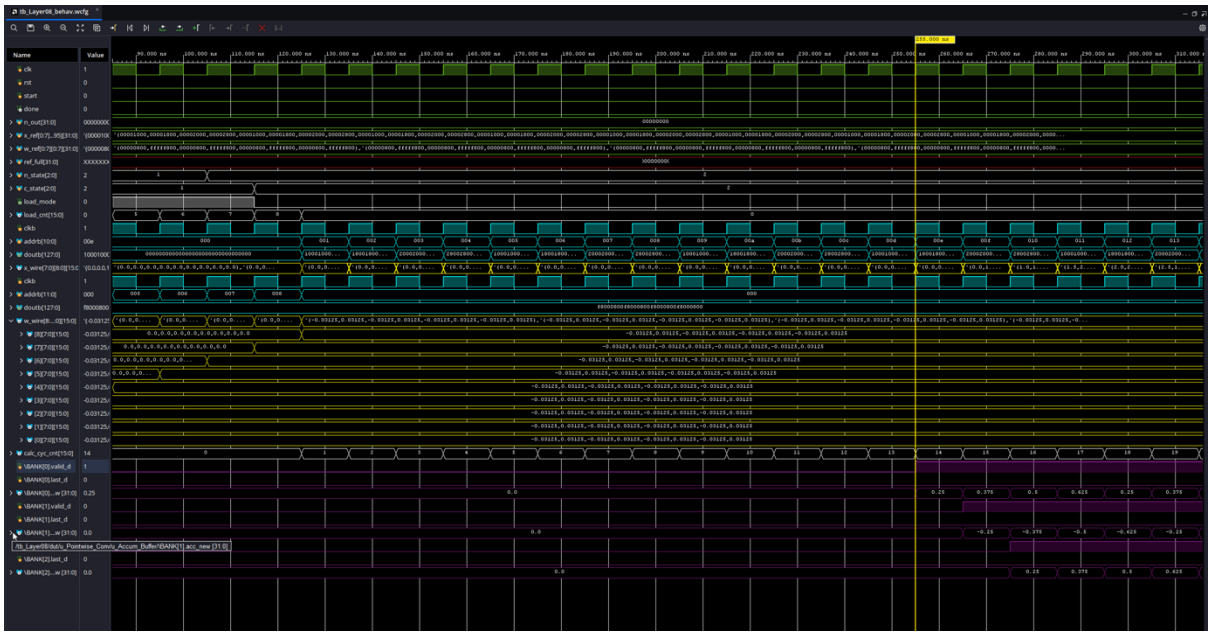


그림 12. 수정 후 파형 (기대값 일치)

8.4 고찰

- 이 옵션은 수백 MHz Fmax를 위한 것으로 100MHz에서는 불필요함.
- 에러/X 없이 "숫자만 조용히 틀리는" 유형이라, 자가채점 TB(전수 비교)와 암산 가능한 디버그 데이터셋이 없었으면 발견이 매우 늦었을 것임.
- 수정 전 캡처는 Docs/보고서/파형캡처(메모리가 FF한번 더 거쳐서 값이 작아짐)/ 폴더, 수정 후는 파형캡처(성공)/ 폴더의 같은 장면을 나란히 비교함.

9. 결론 및 고찰

9.1 결론

- WS 8×8 systolic array 기반 pointwise conv 엔진을 설계하고, 실제 MobileNetV2 Layer08 데이터(BN 폴딩 가중치 24,576개, Layer07 출력 12,544개)로 최종 출력 75,264개 전부를 고정소수점 C 모델과 비트 단위 일치로 검증함.
- 전체 처리는 87,169클럭(0.87ms @100MHz)임.
- 후처리(Post_Proc: bias→ReLU6→재양자화)도 구현·검증까지 완료되어 있으며, 과제 범위(pointwise convolution까지)에 맞춰 제출판에서만 분리함.

9.2 고찰

- 과거 사설 NPU 스터디에서 학습했던 Systolic Array를 직접 적용 시키는 값진 경험이었지만, 그만큼 해당 프로젝트에 적용하는 과정 및 아키텍처를 구상하는 과정에서 시간 소요가 매우 컸음.

- IP 옵션 하나(출력 레지스터)가 에러 없이 값만 틀리는 오답을 만든 사례: "정확히 7/8"이라는 수치의 규칙성에서 역으로 원인(1행 결손)을 특정함. 과제2에서 2629를 소인수분해로 진단한 것과 같은 "결과값 역산 디버깅"의 반복임.

- 디버그 데이터셋: 파형 검증용 데이터는 눈으로 검산 가능해야 함. 쉬운 숫자를 넣어서 하나하나 파형을 분석하기에 쉬웠지만 육안으로 디버깅하기에는 한계가 있음.
  즉 AI를 통해서 테스트 벤치로 검사를 하여 전체 통과라는 결과를 얻었지만, RTL 설계자에게 테스트벤치 작성 역량이 크다는 것을 체감함

9.3 다음 단계

- Post_Proc(bias→ReLU6→재양자화, 검증 완료분) 복원 → 단계 간 중간 버퍼 → Depthwise 엔진(라인버퍼+3×3) → PW 축소(384→64ch, 파라미터 교체 재사용) + residual 덧셈 → Layer08 전체 통합 순으로 진행할 예정임.